INTEGRATED TECHNOLOGIES (AML) LAB

(Course Code: 23UPCSC1L08)

A programming laboratory record submitted to Periyar University, Salem

In partial fulfillment of the requirements for the degree of

MASTER OF COMPUTER APPLICATIONS

By

SRI DEVI M

[Reg. No: U24PG507CAP025]



DEPARTMENT OF COMPUTER SCIENCE

PERIYAR UNIVERSITY

State University - NAAC 'A++' Grade - NIRF Rank 94

State Public University Rank 40 - SDG Institutions Rank Band: 11-50

SALEM – 636 011

(OCTOBER - 2025)

CERTIFICATE

This is to certify that the Programming Laboratory entitled "**INTEGRATED TEECHNOLOGIES (AML) LAB (23UPCSC1L08)"** is a Bonafide record work done by Mr. /Ms._____ Register No: _____ as partial fulfillment of the requirements for the degree of Master of Computer Applications, in the Department of Computer Science, Periyar University, Salem, during the Academic Year 2025-2026.

Staff In-charge Head of the Department

Submitted for the practical examination held on……..……

Internal Examiner External Examiner

CONTENTS

S.NO	DATE	TITLE	PG NO	SIGNATURE
1		Central Tendency Measures		
2		Linear Regression and Multiple Regression		
3		Logistic Regression using sklearn		
4		Binary Classification Model		
5		Nearest Neighbors and Naive Bayes Algorithm		
6		Decision Tree for Classification		
7		K-Means Algorithm		
8		Image Classifier using CNN in TensorFlow/Keras		
9		Autoencoder in TensorFlow/Keras		
10		Simple LSTM using TensorFlow/Keras		

SOURCE CODE:

```
import pandas as pd
import random

dataset={
"marrital_status": random.choices(["Yes","No"],k=600),
"is_graduate": random.choices(["Yes","No"],k=600),
"Income": [random.randint(20000,100000) for _ in range(600)],
"loan_amount":[random.randint(1000,50000) for _ in range(600)],
"term_months": random.choices([12,24,36,48,60], k=600),
"Age": [random.randint(20,60) for _ in range(600)],
"sex": random.choices(["F","M"], k=600),
"approval_status": random.choices(["Yes", "No"], k=600)
}

data=pd.DataFrame(dataset)
data.to_csv(r"C:\Users\CC-Lab55\Documents\dataset.csv", index=False)

print("mean:", round(data['Income'].mean(),3))
print("median:", data['Income'].median())
print("SD", data['Income'].std())
print("var", data['Income'].var())
print("mode", data['Income'].mode()[0])
```

OUTPUT:

```
mean: 60179.108
median: 59977.0
SD 23022.025220036223
var 530013645.23198396
mode 21070
```

SOURCE CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
import matplotlib.pyplot as plt

# Load dataset
dataset =
pd.read_csv("https://raw.githubusercontent.com/selva86/datasets/master/BostonHousing.csv")
dataset.info()
print(dataset.head(4))
# Prepare data
X = dataset.drop('medv', axis=1)
y = dataset['medv']

# Use only 'age' for simple linear regression
x = X[['age']]

# --- Simple Linear Regression ---

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=10)
lr = LinearRegression()
lr.fit(x_train, y_train)
result = lr.predict(x_test)

mse = mean_squared_error(y_test, result)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, result)
print(f"Simple Linear Regression Results:")
print(f"Mean Squared Error: {mse:.2f}")
```

```
print(f"Root Mean Squared Error: {rmse:.2f}")
print(f"R-squared: {r2:.2f}")

plt.scatter(y_test, result)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--',
color='red')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Simple Linear Regression: Actual vs Predicted')
plt.show()

# --- Multiple Linear Regression ---

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=10)
lr = LinearRegression()
lr.fit(x_train, y_train)
result1 = lr.predict(x_test)

mse = mean_squared_error(y_test, result1)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, result1)

print(f"Multiple Linear Regression Results:")
print(f"Mean Squared Error: {mse:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")
print(f"R-squared: {r2:.2f}")

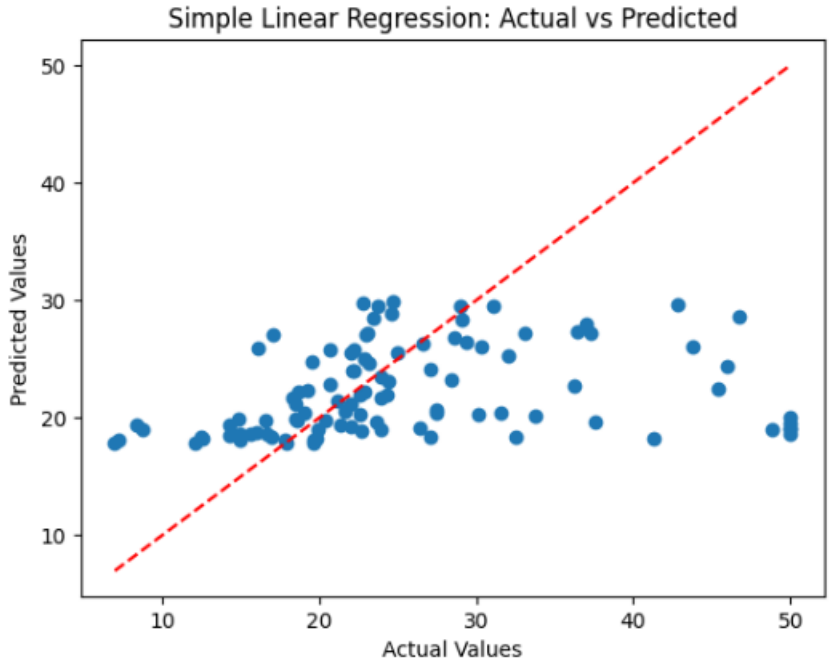
plt.scatter(y_test, result1)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--',
color='red')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Multiple Linear Regression: Actual vs Predicted')
plt.show()
```

OUTPUT:

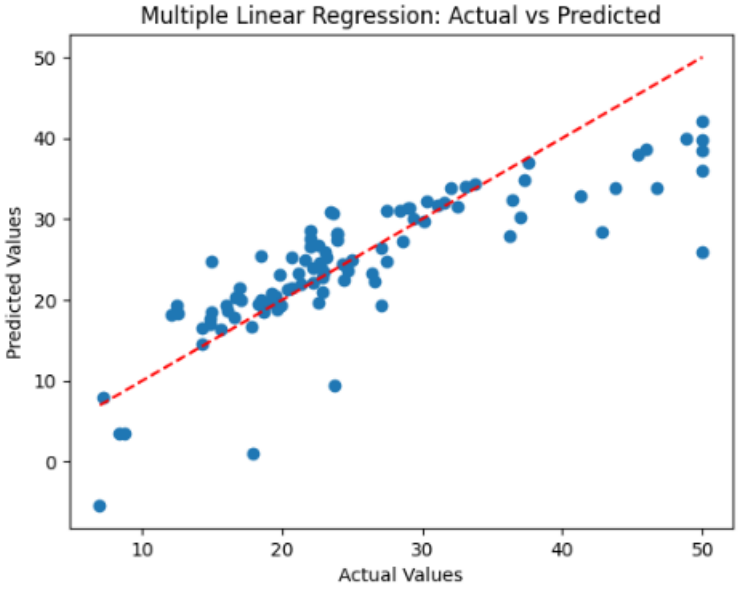
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   crim     506 non-null    float64
 1   zn       506 non-null    float64
 2   indus    506 non-null    float64
 3   chas     506 non-null    int64
 4   nox      506 non-null    float64
 5   rm       506 non-null    float64
```

```
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
      crim    zn  indus  chas    nox     rm   age     dis  rad  tax  ptratio  \
0  0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296     15.3
1  0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242     17.8
2  0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242     17.8
3  0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222     18.7

        b  lstat  medv
0  396.90   4.98  24.0
1  396.90   9.14  21.6
2  392.83   4.03  34.7
3  394.63   2.94  33.4
Simple Linear Regression Results:
Mean Squared Error: 107.93
Root Mean Squared Error: 10.39
R-squared: -0.03
```



```
Multiple Linear Regression Results:
Mean Squared Error: 34.41
Root Mean Squared Error: 5.87
R-squared: 0.67
```

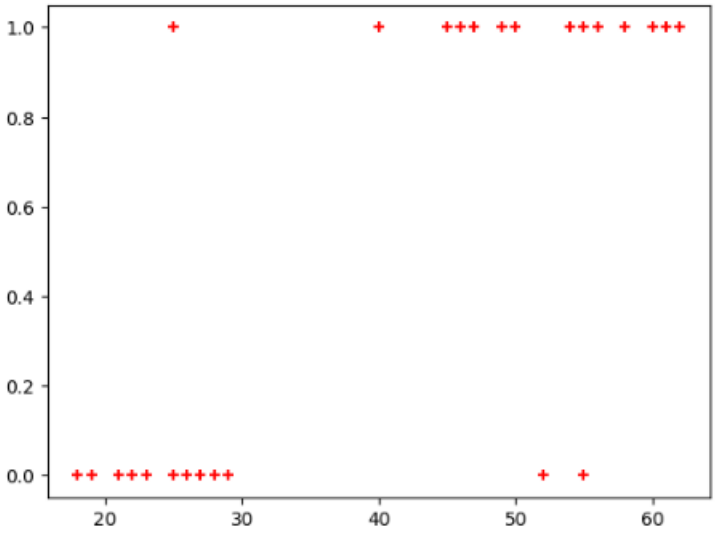


SOURCE CODE:

```
import pandas as pd
from matplotlib import pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
df=pd.read_csv(r"C:\Users\CC-Lab55\Documents\insurance_data.csv")
df.head()

plt.scatter(df.age,df.bought_insurance,marker='+',color='red')
plt.show()
x_train,x_test,y_train,y_test=train_test_split(df[['age']],df.bought_insurance,
train_size  =0.8)
model=LogisticRegression()
model.fit(x_train,y_train)
y_predicted=model.predict(x_test)
model.predict_proba(x_test)
model.score(x_test,y_test)
classification_rep=classification_report(y_test,y_predicted)
print(classification_rep)
```

OUTPUT:



	precision	recall	f1-score	support
0	1.00	1.00	1.00	4
1	1.00	1.00	1.00	2
accuracy			1.00	6
macro avg	1.00	1.00	1.00	6
weighted avg	1.00	1.00	1.00	6

SOURCE CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score,precision_score,f1_score,recall_score

iris=load_iris()
X=iris.data
y=iris.target

y=np.where(y==0,0,1)

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=42)

kernels=['linear','poly','rbf','sigmoid']
metrics={"Accuracy":[],"Precision":[],"Recall":[],"F1 Score":[]}

for k in kernels:
svm=SVC(kernel=k,random_state=42)
svm.fit(X_train,y_train)
y_pred=svm.predict(X_test)

acc=accuracy_score(y_test,y_pred)
prec=precision_score(y_test,y_pred)
rec=recall_score(y_test,y_pred)
f1 =f1_score(y_test,y_pred)

metrics["Accuracy"].append(acc)
metrics["Precision"].append(prec)
metrics["Recall"].append(rec)
metrics["F1 Score"].append(f1)

print(f"\nkernel:{k}")
print(f"Accuracy:{acc:.4f}")
print(f"Precision:{prec:.4f}")
```

```
print(f"Recall:{rec:.4f}")
print(f"F1 Score:{f1:.4f}")

x=np.arange(len(kernels))
width=0.2

plt.bar(x - 0.3, metrics["Accuracy"],width,label='Accuracy')
plt.bar(x - 0.1,metrics["Precision"],width,label='Precision')
plt.bar(x - 0.1,metrics["Recall"],width,label='Recall')
plt.bar(x - 0.1,metrics["F1 Score"],width,label='F1 Score')

plt.xticks(x,kernels)
plt.ylabel("Score")
plt.title("SVM Performance with Different Kernels")
plt.legend()
plt.show()
```

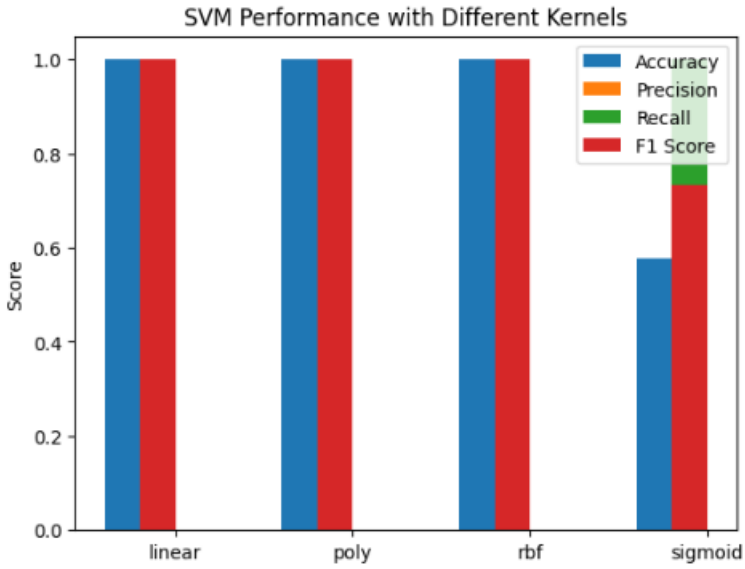
OUTPUT:

```
kernel:linear
Accuracy:1.0000
Precision:1.0000
Recall:1.0000
F1 Score:1.0000

kernel:poly
Accuracy:1.0000
Precision:1.0000
Recall:1.0000
F1 Score:1.0000

kernel:rbf
Accuracy:1.0000
Precision:1.0000
Recall:1.0000
F1 Score:1.0000

kernel:sigmoid
Accuracy:0.5778
Precision:0.5778
Recall:1.0000
F1 Score:0.7324
```



SOURCE CODE:

```
# K Nearest Neighbor Example
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn import metrics
from sklearn.neighbors import KNeighborsClassifier

# Load iris dataset
iris = datasets.load_iris()
df = pd.DataFrame(
    data=np.c_[iris['data'], iris['target']],
    columns=iris['feature_names'] + ['target']
)
print(df.head())

# Visualization
sns.FacetGrid(df, hue="target", height=8).map(
    plt.scatter, "petal length (cm)", "petal width (cm)"
).add_legend()

plt.show()

# KNN model
model = KNeighborsClassifier(n_neighbors=6)
model.fit(iris.data, iris.target)

# Predictions
predicted = model.predict(iris.data)
expected = iris.target
# Accuracy
print("Model Accuracy:", model.score(iris.data, iris.target))
```

```
# Confusion Matrix
print("Confusion Matrix:\n", metrics.confusion_matrix(expected, predicted))

#naive bayes
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn import metrics
from sklearn.naive_bayes import GaussianNB

# Load iris dataset
iris = datasets.load_iris()
df = pd.DataFrame(
    data=np.c_[iris['data'], iris['target']],
    columns=iris['feature_names'] + ['target']
)
print(df.head())

# Visualization
sns.FacetGrid(df, hue="target", height=8).map(
    plt.scatter, "petal length (cm)", "petal width (cm)"
).add_legend()

plt.show()
model=GaussianNB()


model.fit(iris.data, iris.target)

# Predictions
predicted = model.predict(iris.data)
expected = iris.target

# Accuracy
print("Model Accuracy:", model.score(iris.data, iris.target))

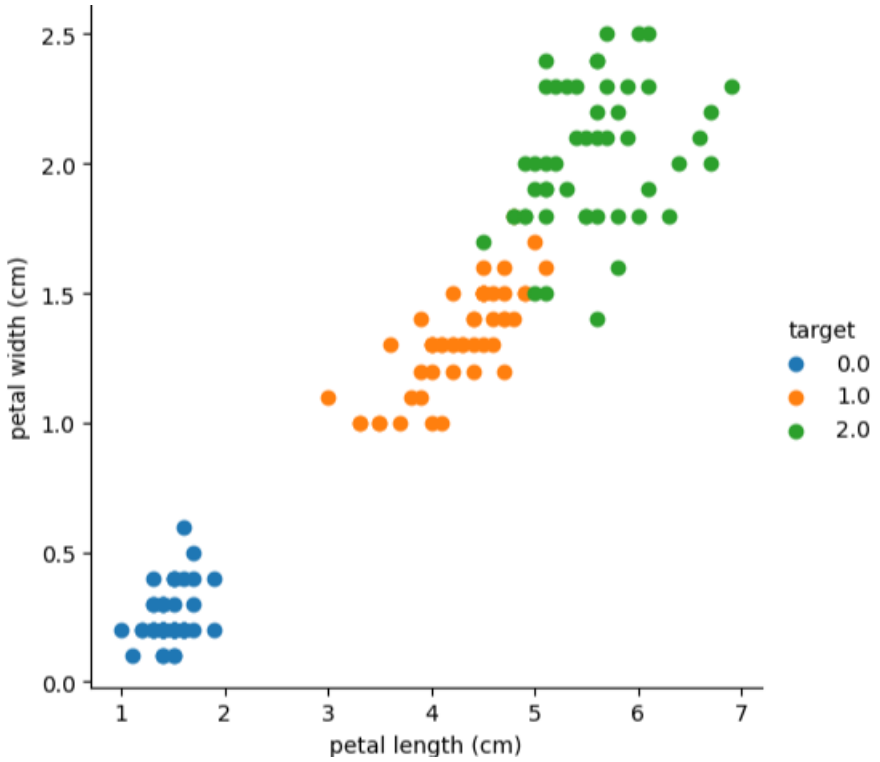
# Confusion Matrix
print("Confusion Matrix:\n", metrics.confusion_matrix(expected, predicted))
```

OUTPUT:

K Nearest Neighbor

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0     0.0
1     0.0
2     0.0
3     0.0
4     0.0
```

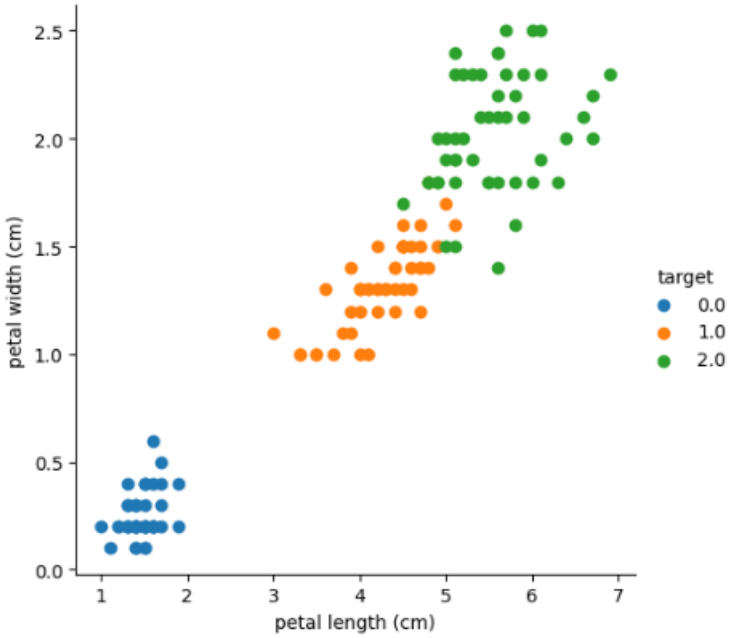



```
Model Accuracy: 0.9733333333333334
Confusion Matrix:
 [[50  0  0]
 [ 0 48  2]
 [ 0  2 48]]
```

Naive bayes

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0     0.0
1     0.0
2     0.0
3     0.0
4     0.0
```




```
Model Accuracy: 0.96
Confusion Matrix:
 [[50  0  0]
 [ 0 47  3]
 [ 0  3 47]]
```

SOURCE CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report

dataset=pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-
data/refs/heads/master/iris.csv')
x=dataset.drop('species',axis=1)
y=dataset['species']


x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2, random_state=42)

dt=DecisionTreeClassifier(criterion='entropy', splitter='random', max_depth=3)
dt.fit(x_train, y_train)
y_pred=dt.predict(x_test)

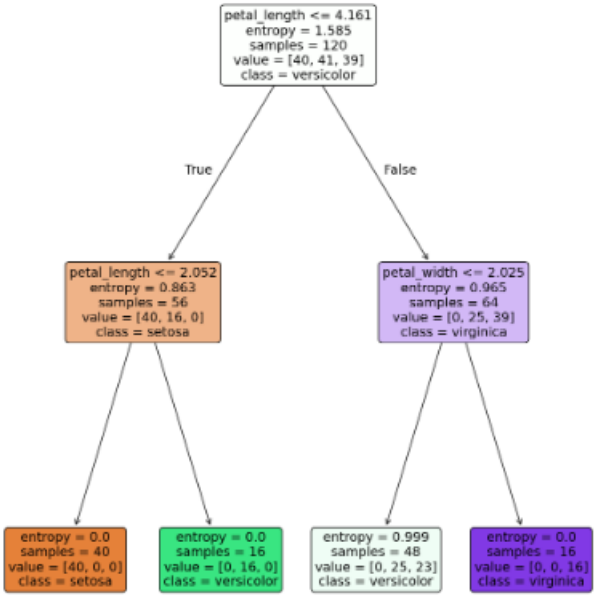
acc=accuracy_score(y_test,y_pred)
acc

class_names_list=y.unique().astype(str).tolist()
feature_names_list=x.columns.tolist()

plt.figure(figsize=(12,15))
plot_tree(dt, filled=True,feature_names=feature_names_list,
class_names=class_names_list,rounded=True)
plt.title("Decision Tree Visualization")
plt.show()
```

OUTPUT:

Decision Tree Visualization



SOURCE CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Load
dataset = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv')
x = dataset.drop('species', axis=1).values

# Corrected typo: use the same variable name
kmeans = KMeans(n_clusters=3, random_state=42)  # Changed from 30 to 3 for iris
kmeans.fit(x)

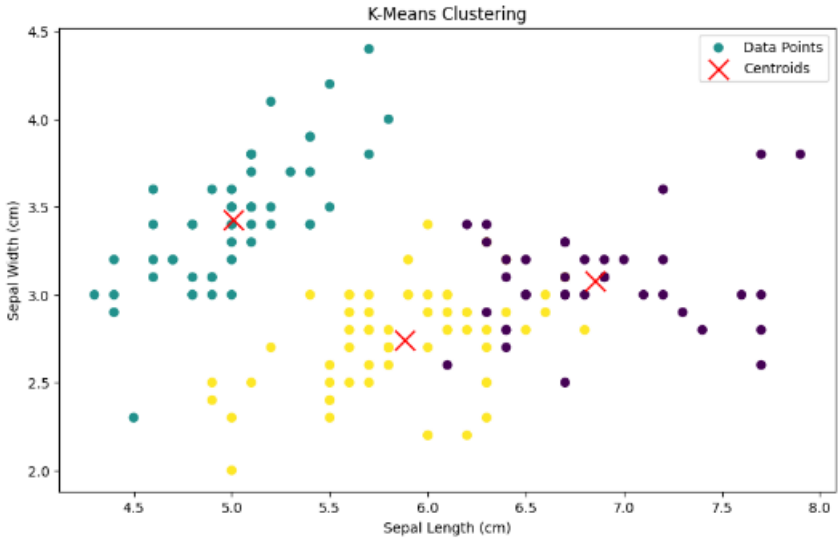
cl = kmeans.labels_
ce = kmeans.cluster_centers_

print("type of x:", type(x), "shape of x:", x.shape)
print("type of cl:", type(cl), "shape of cl:", cl.shape)
print("type of ce:", type(ce), "shape of ce:", ce.shape)

# Plotting
plt.figure(figsize=(10, 6))
plt.scatter(x[:, 0], x[:, 1], c=cl, cmap='viridis', label='Data Points')
plt.scatter(ce[:, 0], ce[:, 1], c='red', marker='x', s=200, label='Centroids')
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.title("K-Means Clustering")
plt.legend()
plt.show()
```

OUTPUT:

```
type of x: <class 'numpy.ndarray'> shape of x: (150, 4)
type of cl: <class 'numpy.ndarray'> shape of cl: (150,)
type of ce: <class 'numpy.ndarray'> shape of ce: (3, 4)
```



SOURCE CODE:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

# Load and preprocess data
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
y_train = to_categorical(y_train, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)

# Clear any previous session
tf.keras.backend.clear_session()

# Build CNN model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
history = model.fit(x_train, y_train,
                    epochs=5,
```

```
                batch_size=64,
                validation_split=0.1)

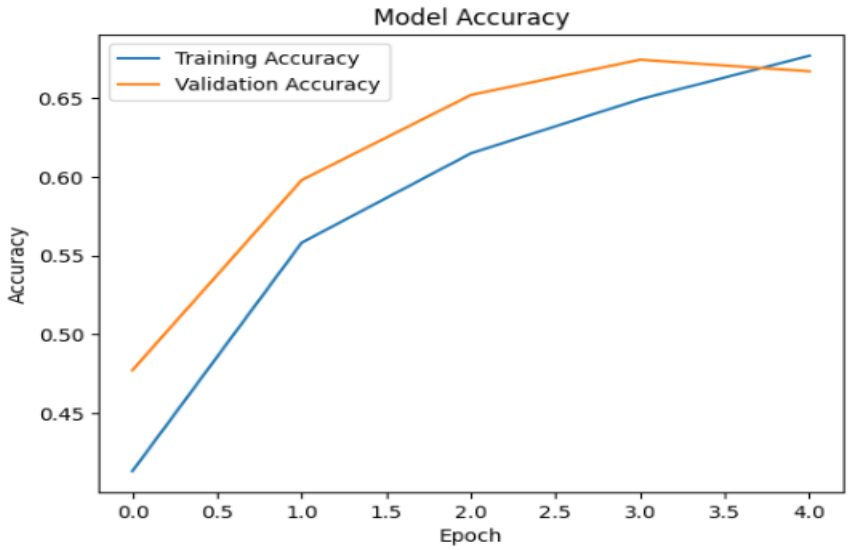
# Evaluate the model on test data
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)

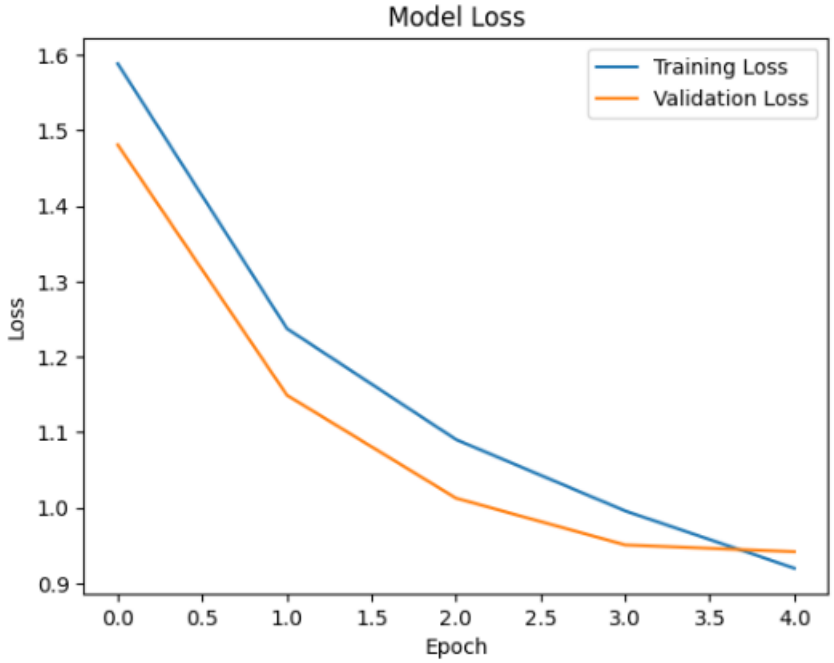
# Plot training and validation accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title("Model Accuracy")
plt.show()

# Plot training and validation loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title("Model Loss")
plt.show()
```

OUTPUT:

```
Epoch 1/5
704/704 [==============================] - 82s 114ms/step - loss: 1.5885 - accuracy: 0.4134 - val_loss: 1.4809 - val_accuracy: 0.4774
Epoch 2/5
704/704 [==============================] - 77s 110ms/step - loss: 1.2374 - accuracy: 0.5583 - val_loss: 1.1491 - val_accuracy: 0.5980
Epoch 3/5
704/704 [==============================] - 79s 112ms/step - loss: 1.0903 - accuracy: 0.6150 - val_loss: 1.0126 - val_accuracy: 0.6520
Epoch 4/5
704/704 [==============================] - 75s 107ms/step - loss: 0.9957 - accuracy: 0.6493 - val_loss: 0.9506 - val_accuracy: 0.6744
Epoch 5/5
704/704 [==============================] - 78s 110ms/step - loss: 0.9198 - accuracy: 0.6769 - val_loss: 0.9416 - val_accuracy: 0.6670
313/313 [==============================] - 6s 19ms/step - loss: 0.9619 - accuracy: 0.6650
Test accuracy: 0.6650000214576721
```





SOURCE CODE:

```
import tensorflow as tf
from tensorflow.keras import layers,models
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt

(x_train,_),(x_test,_)=mnist.load_data()
x_train=x_train.reshape((60000,784)).astype("float32")/255.0
x_test=x_test.reshape((10000,784)).astype("float32")/255.0

#create autoencoder model
encoding_dim=32
input_img=tf.keras.Input(shape=(784,))

#encoder
encoded=layers.Dense(encoding_dim,activation='relu')(input_img)

#decoder
decoded=layers.Dense(784,activation='sigmoid')(encoded)

#autoencoder model
autoencoder=models.Model(input_img,decoded)

#compile model
autoencoder.compile(optimizer='adam',loss='binary_crossentropy')

#train autoencoder
history=autoencoder.fit(
x_train,x_train,
epochs=5,
batch_size=256,
shuffle=True,
validation_data=(x_test,x_test)
)

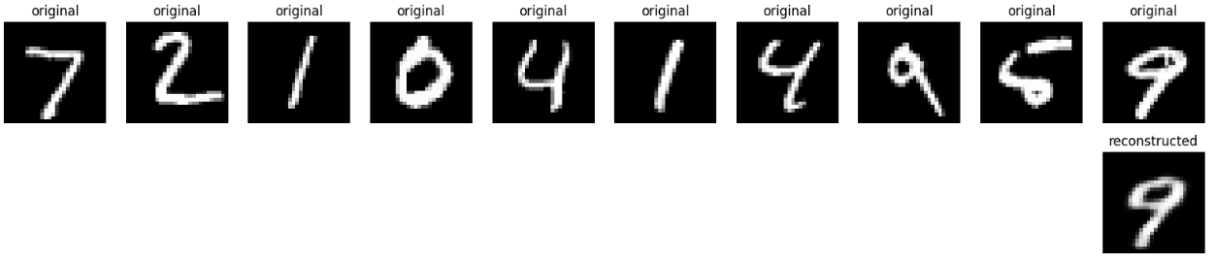
#reconstruct test image
decoded_imgs=autoencoder.predict(x_test)
```

```
#display original vs reconstructed image
n=10
plt.figure(figsize=(20,4))
for i in range(n):
plt.subplot(2,n,i+1)
plt.imshow(x_test[i].reshape(28,28),cmap='gray')
plt.title("original")
plt.axis("off")

#reconstructed
plt.subplot(2,n,i+1+n)
plt.imshow(decoded_imgs[i].reshape(28,28),cmap='gray')
plt.title("reconstructed")
plt.axis("off")
plt.show()
```

OUTPUT:

```
Epoch 1/5
235/235 [==============================] - 4s 13ms/step - loss: 0.2746 - val_loss: 0.1898
Epoch 2/5
235/235 [==============================] - 3s 14ms/step - loss: 0.1715 - val_loss: 0.1539
Epoch 3/5
235/235 [==============================] - 4s 15ms/step - loss: 0.1449 - val_loss: 0.1349
Epoch 4/5
235/235 [==============================] - 3s 14ms/step - loss: 0.1298 - val_loss: 0.1227
Epoch 5/5
235/235 [==============================] - 3s 14ms/step - loss: 0.1195 - val_loss: 0.1141
313/313 [==============================] - 1s 3ms/step
```



SOURCE CODE:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequence
import matplotlib.pyplot as plt

# Load and preprocess the data
max_features = 10000 # Number of words to consider as features
maxlen = 100 # Maximum length of sequences
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)

# Create the Simple LSTM model
embedding_dim = 128
model = models.Sequential()
model.add(layers.Embedding(max_features, embedding_dim,
input_length=maxlen))
model.add(layers.LSTM(128))
model.add(layers.Dense(1, activation='sigmoid'))

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(x_train, y_train, epochs=5, batch_size=32, validation_split=0.1)

# Evaluate the model on test data
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)

# Plot training and validation loss
plt.figure(figsize=(8, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
plt.legend()
plt.show()

# Plot training and validation accuracy
plt.figure(figsize=(8, 6))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

OUTPUT:

```
Epoch 1/3
704/704 ━━━━━━━━━━━━━━━━━━━━ 203s 288ms/step - accuracy: 0.8613 - loss: 0.3280 - val_accuracy: 0.8380 - val_loss: 0.3661
Epoch 2/3
704/704 ━━━━━━━━━━━━━━━━━━━━ 235s 249ms/step - accuracy: 0.9111 - loss: 0.2268 - val_accuracy: 0.8236 - val_loss: 0.3915
Epoch 3/3
704/704 ━━━━━━━━━━━━━━━━━━━━ 224s 279ms/step - accuracy: 0.9407 - loss: 0.1602 - val_accuracy: 0.8304 - val_loss: 0.5067
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 95s 120ms/step - accuracy: 0.8232 - loss: 0.5192
Test accuracy: 0.823199987411499
```

